



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

x

CHAPTER 3

Pengenalan TensorFlow, PyTorch, JAX, dan Keras

Satu lapis Dense yang sama, ditulis empat kali – supaya perbedaan rancangan keempat kerangka kerja itu terlihat, bukan sekadar didengar.

Durasi 2,5 jam

Pengajar Rahman Indra Kesuma, S.Kom., M.Cs.

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 3

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Menyebut **tiga kemampuan** yang dimiliki semua kerangka kerja modern, dan apa yang membedakan ketiganya di luar itu.
- 2 Menghitung gradien dengan **GradientTape**, `.backward()`, dan `jax.grad()`, dan menjelaskan mengapa ketiganya berbeda bentuk.
- 3 Membedakan **stateful imperatif** (TF, PyTorch) dari **stateless fungsional** (JAX), dan menyebut akibatnya pada penulisan lingkaran pelatihan.
- 4 Mengganti backend Keras 3 tanpa mengubah satu baris pun kode model.
- 5 Menulis **Layer kustom** dengan `build()` dan `call()`, lalu memakainya lewat `compile()` dan `fit()`.

BUKU

[Bab 3 — teks penuh](#)

NOTEBOOK

[01_tiga_kerangka_berdampingan.ipynb](#)

NOTEBOOK

[02_keras3_ganti_backend.ipynb](#)

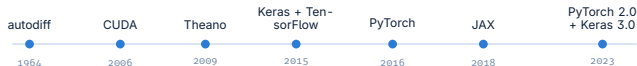
NOTEBOOK

[03_layer_kustom_dan_fit.ipynb](#)

REPO

[Notebook resmi penulis](#)

Bagaimana kita sampai di sini



Pertengahan 2016: lebih dari separuh pengguna TensorFlow mengaksesnya lewat Keras

Autodiff sudah ada sejak 1964; yang baru pada 2009 adalah menggabungkannya dengan komputasi GPU.

Automatic differentiation

Untuk sebarang fungsi terdiferensialkan yang Anda tulis.

Komputasi tensor

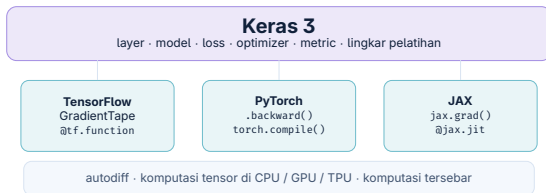
Di CPU, GPU, dan perangkat keras khusus seperti TPU.

Komputasi tersebar

Antar-perangkat dan antar-mesin.

Ketiganya dimiliki **semua** kerangka kerja besar. Jadi memilih kerangka kerja **bukan soal kemampuan**, melainkan soal gaya penulisan, ekosistem, dan kecepatan.

Keras di atas, tiga mesin di bawah



Keras memerlukan sebuah backend; NumPy bisa dipasang tetapi tidak bisa melatih, sebab tidak punya API gradien.

Keras itu seperti perangkat bangunan pracetak, sedangkan TensorFlow, PyTorch, dan JAX adalah bahan mentahnya.

Chollet & Watson, bab 3

01

TensorFlow

Tensor kekal, Variable untuk keadaan, GradientTape untuk gradien.

Tensor kekal, Variable yang bisa diubah

tensor dan variable di TensorFlow

PYTHON

```
import tensorflow as tf

tf.ones(shape=(2, 1))
tf.zeros(shape=(2, 1))
tf.constant([1.0, 2.0])          # KEKAL - tidak bisa ditugasi ulang

v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
v.assign(tf.ones((3, 1)))        # ganti seluruh nilainya
v[0, 0].assign(3.0)              # ganti sebagiannya
v.assign_add(tf.ones((3, 1)))    # += yang efisien

a = tf.ones((2, 2))
e = tf.matmul(a, tf.square(a))
f = tf.concat((a, e), axis=0)     # perhatikan: 'axis'
```

Bedanya penting: tensor TensorFlow itu **konstanta yang kekal**. Untuk parameter yang harus berubah saat dilatih, Anda **wajib** memakai `tf.Variable`.

GradientTape dan kompilasi graf

```
gradien PYTHON  
  
input_var = tf.Variable(3.0)  
with tf.GradientTape() as tape:  
    result = tf.square(input_var)  
gradient = tape.gradient(result, input_var)  
  
# konstanta harus 'ditonton' dulu  
c = tf.constant(3.0)  
with tf.GradientTape() as tape:  
    tape.watch(c)  
    result = tf.square(c)  
gradient = tape.gradient(result, c)
```

Kekuatan

Cepat lewat mode graf dan XLA - lengkap sekali (tensor string, ragged tensor) - `tf.data` unggul untuk pra-pemrosesan - ekosistem produksi paling matang (TFX, TF-Serving, TFLite).

```
kompilasi PYTHON  
  
@tf.function  
def dense(inputs, W, b):  
    return tf.nn.relu(tf.matmul(inputs, W) + b)  
  
# XLA: lebih agresif, kompilasi  
# pertama lebih lama  
@tf.function(jit_compile=True)  
def dense(inputs, W, b):  
    return tf.nn.relu(tf.matmul(inputs, W) + b)
```

Kelemahan

API sangat luas dengan ribuan operasi - di beberapa tempat menyimpang dari NumPy - dukungan di Hugging Face kalah dari PyTorch.

02

PyTorch

Tensor bisa diubah, `.backward()` mengisi `.grad`, eager sebagai bawaan.

Gaya PyTorch: tanpa pita, ada .grad

tensor, parameter, gradien

PYTHON

```
import torch                                # paketnya 'torch', bukan 'pytorch'

x = torch.zeros(size=(2, 1))
x[0, 0] = 1.0                               # BISA ditugasi - beda dari TensorFlow

p = torch.nn.parameter.Parameter(data=x)     # penanda: ini keadaan terlatih

f = torch.cat((torch.ones((2, 2)), x), dim=0) # perhatikan: 'dim', bukan 'axis'

input_var = torch.tensor(3.0, requires_grad=True)
result = torch.square(input_var)
result.backward()                            # mengisi input_var.grad
print(input_var.grad)

input_var.grad = None                       # WAJIB: gradien menumpuk kalau tidak dinolkan
```

Baris terakhir itu sumber bug klasik. Panggilan `.backward()` berikutnya **menjumlahkan** gradien baru ke yang lama, bukan menggantinya.

Module, optimizer, dan mantra tiga baris

pola pelatihan PyTorch

PYTHON

```
class LinearModel(torch.nn.Module):
    def __init__(self, input_dim, output_dim):
        super().__init__()
        self.W = torch.nn.Parameter(torch.rand(input_dim, output_dim))
        self.b = torch.nn.Parameter(torch.zeros(output_dim))

    def forward(self, inputs):
        return torch.matmul(inputs, self.W) + self.b

model = LinearModel(2, 1)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

def training_step(inputs, targets):
    predictions = model(inputs)
    loss = mean_squared_error(targets, predictions)
    loss.backward()      # 1. hitung gradien
    optimizer.step()     # 2. perbarui bobot
    model.zero_grad()    # 3. nolkan, siap batch berikutnya
    return loss
```

Kekuatan

Eager sebagai bawaan – pengawakutuan paling mudah · dukungan kelas satu di Hugging Face, dan itulah pendorong adopsi terbesarnya.

Kelemahan

API tidak konsisten (`axis` kadang jadi `dim`) · menurut penulis, **paling lambat** di antara yang besar · `torch.compile()` masih penuh kasus tepi dan jarang dipakai.

03

JAX

Fungsi tanpa keadaan. Gradien sebagai transformasi fungsi.

Tanpa keadaan – termasuk bilangan acaknya

array, kunci acak, pembaruan

PYTHON

```
import jax
from jax import numpy as jnp

jnp.ones(shape=(2, 1))          # API NumPy, tanpa penyimpangan

# Tidak ada keadaan acak global: kunci diberikan secara eksplisit
seed_key = jax.random.key(123)
jax.random.normal(seed_key, shape=(3,))    # kunci sama -> nilai sama, selalu
key1, key2 = jax.random.split(seed_key)    # cara membuat kunci baru

# Array kekal: perbarui dengan menghasilkan array baru
x = jnp.array([1, 2, 3], dtype="float32")
new_x = x.at[0].set(10)
```

Kelihatannya merepotkan, dan memang. Imbalannya: perhitungan jadi **bisa diparalelkan otomatis tanpa sinkronisasi**, dan hasilnya deterministik – dua hal yang menentukan pada skala besar.

jax.grad(): gradien sebagai transformasi fungsi

lingkar pelatihan JAX yang utuh

PYTHON

```
def model(inputs, W, b):
    return jnp.matmul(inputs, W) + b

def compute_loss(state, inputs, targets):
    W, b = state
    predictions = model(inputs, W, b)
    return jnp.mean(jnp.square(targets - predictions))

grad_fn = jax.value_and_grad(compute_loss) # fungsi -> fungsi gradien

@jax.jit
def training_step(inputs, targets, W, b):
    loss, grads = grad_fn((W, b), inputs, targets)
    grad_W, grad_b = grads
    W = W - grad_W * 0.1
    b = b - grad_b * 0.1
    return loss, W, b # keadaan WAJIB dikembalikan
```

KEBUTUHAN

Gradien saja
Rugi **dan** gradien sekaligus
Ada keluaran sampingan

PEMANGGILANNYA

`jax.grad(f)`
`jax.value_and_grad(f)` – lebih hemat
`jax.value_and_grad(f, has_aux=True)`

Perhatikan baris terakhir: `training_step` mengembalikan W dan b. Tidak ada yang berubah di tempat. Inilah harga dan sekaligus keuntungan gaya fungsional.

Ketiganya berdampingan

	TENSORFLOW	PYTORCH	JAX
Paradigma	Imperatif berkeadaan	Imperatif berkeadaan	Fungsional tanpa keadaan
Tensor	Kekal (<code>Variable</code> untuk keadaan)	Bisa diubah	Kekal (<code>.at[].set()</code>)
Gradien	<code>GradientTape</code>	<code>.backward()</code> → <code>.grad</code>	<code>jax.grad()</code> (transformasi)
Kompilasi	<code>@tf.function</code> , XLA	<code>@torch.compile</code> (Dynamo)	<code>@jax.jit</code> (XLA)
Eksekusi	Eager + graf	Eager (bawaan)	Eager + JIT
Awak-kutu	Lebih sulit di mode graf	Paling mudah	Sulit (fungsional + JIT)
Ekosistem	Perkakas produksi	Hugging Face, riset	Riset, skala Google

Peringkat kecepatan di bab ini – JAX tercepat, PyTorch terlambat, selisih 20-30% dan sampai 3-5x pada model besar – adalah **penilaian penulis buku**, bukan tolok ukur yang dijalankan kelas ini. Perlakukan sebagai petunjuk arah, dan ukur sendiri untuk beban kerja Anda.

04

Keras

Satu kode model, tiga mesin di belakangnya.

Mengganti backend tanpa menyentuh model

cara 1 – variabel lingkungan

PYTHON

```
import os
os.environ["KERAS_BACKEND"] = "jax"

import keras          # HARUS setelah baris di atas
print(keras.backend.backend())
```

OUTPUT

jax

cara 2 – ~/.keras/keras.json

JSON

```
{
  "floatx": "float32",
  "epsilon": 1e-07,
  "backend": "tensorflow",
  "image_data_format": "channels_last"
}
```

Urutannya tidak bisa ditawar: `os.environ[...]` harus dijalankan **sebelum** `import keras` **yang pertama**. Setelah Keras terimpor, menggantinya tidak berpengaruh, dan ini kebingungan nomor satu di praktikum.

Penulis menyarankan **JAX** untuk unjuk kerja terbaik, tetapi kode Keras yang sama berjalan di ketiganya.

Layer: satuan bangunan, dan penyimpulan bentuk otomatis

listing 3.22 ~ Layer kustom

PYTHON

```
import keras

class SimpleDense(keras.Layer):
    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = activation

    def build(self, input_shape):
        # dipanggil sekali, saat masukan pertama tiba
        batch_dim, input_dim = input_shape
        self.W = self.add_weight(shape=(input_dim, self.units),
                                initializer="random_normal")
        self.b = self.add_weight(shape=(self.units,), initializer="zeros")

    def call(self, inputs):
        y = keras.ops.matmul(inputs, self.W) + self.b
        return self.activation(y) if self.activation is not None else y

my_dense = SimpleDense(units=32, activation=keras.ops.relu)
output = my_dense(keras.ops.ones(shape=(2, 784)))
print(output.shape)
```

OUTPUT

(2, 32)

Karena `build()` menerima `input_shape`, Anda **tidak perlu menyebutkan ukuran masukan** saat menyusun model. Itulah sebabnya `Sequential` di bab 2 hanya menyebut jumlah unit.

compile(), fit(), dan data validasi

listing 3.26-3.29 – alur lengkap

PYTHON

```
model = keras.Sequential([keras.layers.Dense(1)])

model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.BinaryAccuracy()]
)

history = model.fit(
    training_inputs, training_targets,
    epochs=5, batch_size=16,
    validation_data=(val_inputs, val_targets),
)

print(history.history.keys())

loss_and_metrics = model.evaluate(val_inputs, val_targets, batch_size=128)
predictions = model.predict(new_inputs, batch_size=128)
```

Tujuan machine learning bukan memperoleh model yang bekerja baik pada data latih – melainkan model yang bekerja baik secara umum, terutama pada data yang belum pernah ditemuinya.

Chollet & Watson, bab 3

Karena itu **metric** dipantau tetapi **tidak dioptimalkan**; yang dioptimalkan hanya loss. Membingungkan keduanya adalah kesalahan yang mahal, dan bab 5-6 kembali ke sini.

Yang wajib terbawa dari bab 3

- 1 Semua kerangka kerja besar memberi **autodiff, komputasi tensor di GPU/TPU, dan komputasi tersebar**. Sisanya soal gaya.
- 2 **TensorFlow** – tensor kekal + `Variable`, `GradientTape`, ekosistem produksi terkuat.
- 3 **PyTorch** – tensor bisa diubah, `.backward()`, eager, raja Hugging Face. Jangan lupa `zero_grad()`.
- 4 **JAX** – fungsional tanpa keadaan, `jax.grad()` sebagai transformasi, kunci acak eksplisit, tercepat menurut penulis.
- 5 **Keras 3** duduk di atas ketiganya. Ganti backend lewat `KERAS_BACKEND` **sebelum** `import keras`.
- 6 `Layer` dengan `build()` + `call()` adalah satuan bangunan segalanya; bentuk masukan disimpulkan sendiri.

NOTEBOOK

[01_tiga_kerangka_berdampingan.ipynb](#)

BAB BERIKUT

[Bab 4 — Klasifikasi dan regresi](#)

KODE BUKU

[notebook resmi bab 3](#)